

# Sumario

|                                                                                                 |    |
|-------------------------------------------------------------------------------------------------|----|
| 1. Introducción a la gestión concurrente de múltiples procesos en Linux.....                    | 2  |
| 2. Planificación de procesos.....                                                               | 4  |
| 3. Colas de planificación de procesos.....                                                      | 5  |
| 3.1 Cola de procesos Listos o Preparados (Ready Queue).....                                     | 5  |
| 3.2 Colas de Dispositivos (Device Queues).....                                                  | 6  |
| 3.3 Cola de Suspendidos (Wait/Suspend Queues).....                                              | 7  |
| 4. Políticas de planificación.....                                                              | 9  |
| 4.1 Primero en llegar primero en ser servido, o FCFS (del inglés, First Come First Served)..... | 9  |
| 4.1.1 Actividad.....                                                                            | 12 |
| 4.2 Primero el más corto, o SJF (del inglés, <i>Shortest Job First</i> ).....                   | 13 |
| 4.2.1 Actividad: Planificación SRTN.....                                                        | 16 |
| 4.3 Por turnos, o RR (del inglés, <i>Round Robin</i> ).....                                     | 17 |
| 4.3.1 Actividad: Planificación por turnos.....                                                  | 19 |
| 4.4 Planificación por prioridad.....                                                            | 20 |
| 5. Comandos linux que interactúan con el planificador de procesos.....                          | 22 |
| 5.1 Comando "ps" - Visualización de Procesos.....                                               | 22 |
| 5.1.1 Actividades.....                                                                          | 23 |
| 5.1.2 Actividades.....                                                                          | 24 |
| 5.2 Comando "kill" - Terminación de Procesos.....                                               | 25 |
| 5.1.3 Actividades.....                                                                          | 26 |
| 5.3. Comando "top" - Monitoreo en Tiempo Real.....                                              | 27 |
| 5.3.1 Actividades.....                                                                          | 28 |
| 5.4 Comando pstree.....                                                                         | 29 |

|                                                       |    |
|-------------------------------------------------------|----|
| 5.4.1 Actividades.....                                | 30 |
| 5.5. pgrep.....                                       | 31 |
| 5.5.1 Actividades.....                                | 32 |
| 5.6 nice y renice.....                                | 33 |
| 5.6.1 Actividades.....                                | 34 |
| 5.7 Modificación de la política de planificación..... | 34 |

# **1. Introducción a la gestión concurrente de múltiples procesos en Linux**

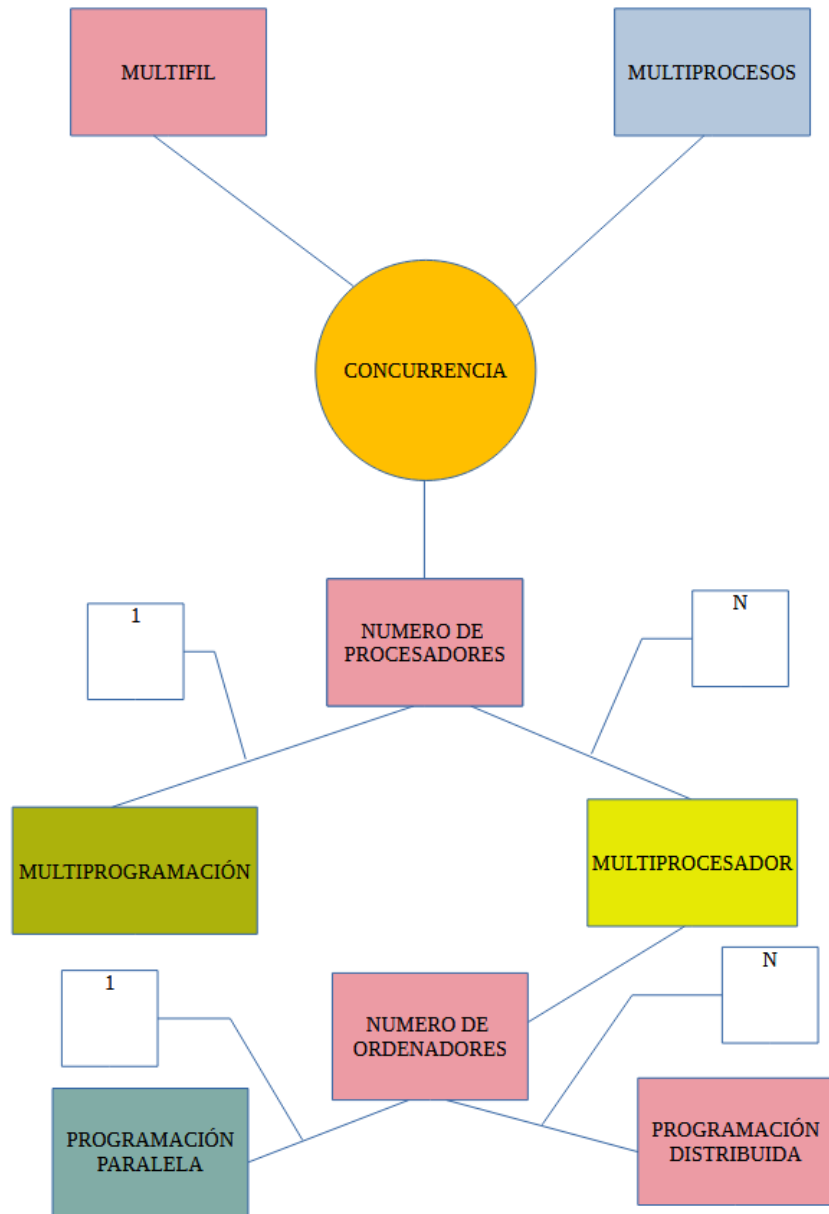
En Linux, la gestión de procesos es fundamental para mantener un sistema eficiente y seguro.

Los procesos son unidades de ejecución que pueden ser aplicaciones, servicios del sistema o incluso comandos simples. A lo largo de este curso, aprenderás a administrar procesos en Linux y entenderás cómo funcionan de manera concurrente en el sistema.

Concurrencia:

Se define como la situación en la que dos o más procesos o hilos se ejecutan en un equipo de forma simultanea.

Tipos de concurrencia:

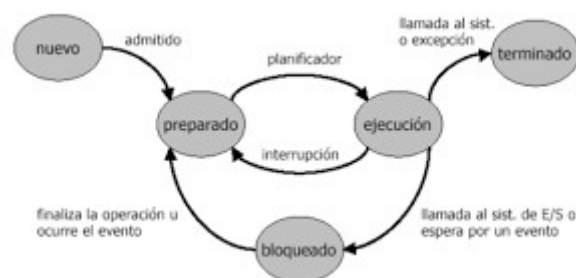


## 2. Planificación de procesos

Una de las claves para que un sistema multiprogramado sea eficaz es la *Planificación de procesos*, que consiste en ir asignando procesos al procesador (o procesadores / núcleos) a lo largo del tiempo, de forma que se cumplan los objetivos en varios aspectos:

- *Rendimiento*: Trata de maximizar el número de acciones que se completan en un plazo de tiempo determinado.
- *Tiempo de respuesta*: El sistema debe responder a las solicitudes de los usuarios en un tiempo adecuado.
- *Equidad*: Todos los procesos deben ser considerados según sus características.
- *Eficiencia*: Se debe aspirar a que el procesador esté activo constantemente.

Como cabe esperar, el módulo del sistema operativo que se encarga de esta tarea se denomina *Planificador* (en inglés, *Scheduler*).



Según el diseño del sistema operativo, el *Planificador* utilizará unos criterios u otros para llevar a cabo su tarea. Estos criterios reciben el nombre de *Algoritmos de Planificación* (o también, *Políticas de Planificación*).

### Nota

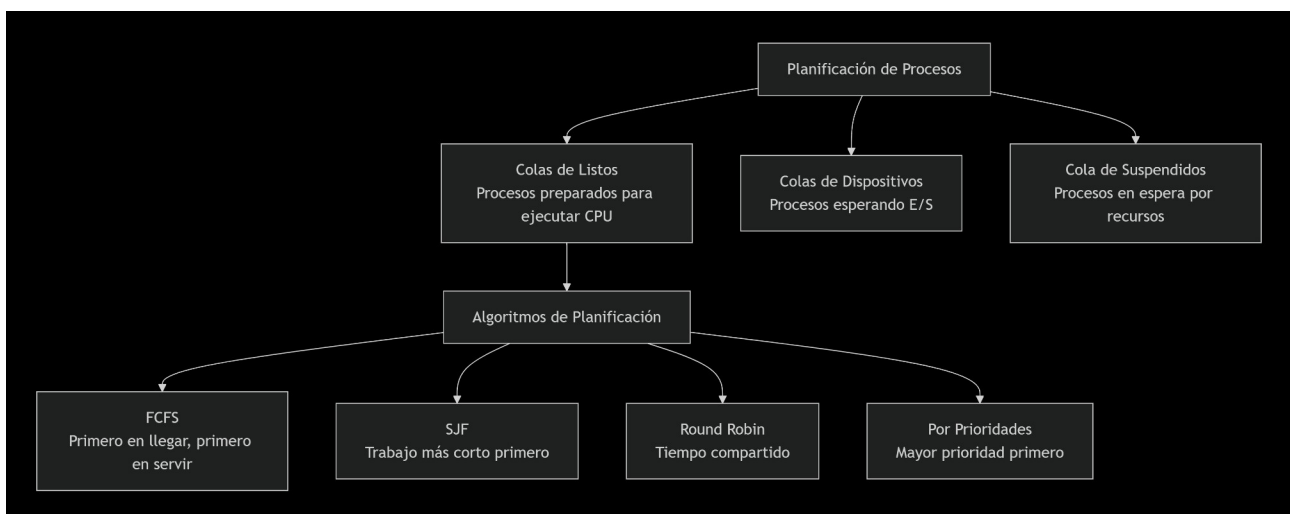
Una política de planificación es **no apropiativa** (en inglés, *non-preemptive*) cuando, una vez que un proceso toma el control del procesador, no lo abandona hasta que termina o hasta que se bloquea. Será **apropiativa** (en inglés, *preemptive*) cuando el sistema puede interrumpir el proceso para ejecutar otro diferente.

### 3. Colas de planificación de procesos

El esquema siguiente representa los distintos tipos de colas en los que se puede encontrar un proceso:

#### 3.1 Cola de procesos Listos o Preparados (Ready Queue)

- Función:** Procesos listos para ejecución esperando CPU
- Características:**
  - Estructura FIFO (First-In, First-Out) o prioridades
  - El planificador selecciona procesos de esta cola



## **3.2 Colas de Dispositivos (Device Queues)**

**Función:** Procesos esperando por dispositivos de E/S

- **Características:**

- Una cola por cada dispositivo
- Los procesos permanecen bloqueados hasta que el dispositivo esté disponible
- Se gestionan mediante el subsistema de E/S

### 3.3 Cola de Suspendidos (Wait/Suspend Queues)

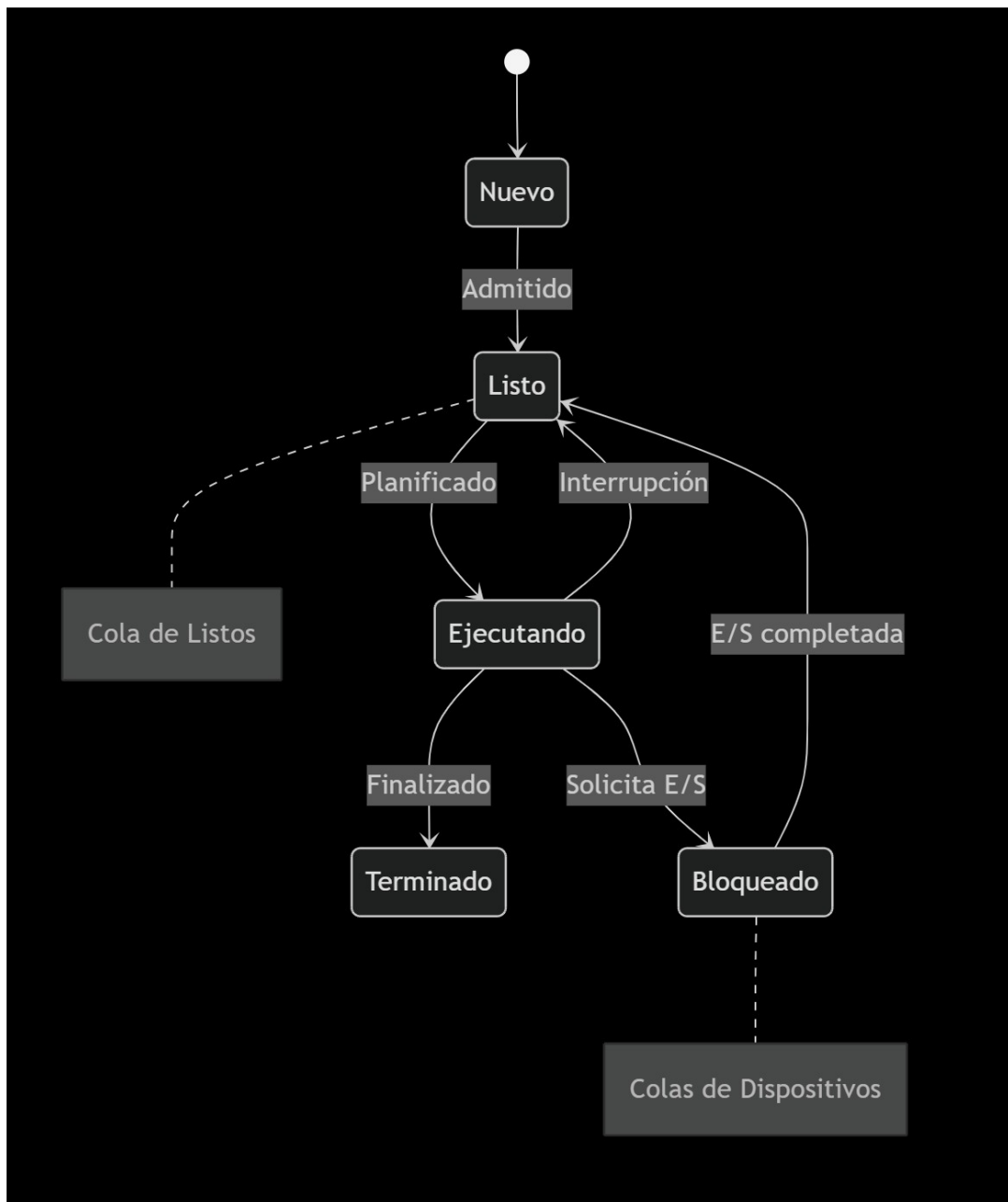
- **Función:** Procesos esperando por eventos específicos

- **Características:**

- Espera por semáforos, señales o eventos

- Procesos en estado de bloqueo

- Se reactivan cuando ocurre el evento esperado



## 4. Políticas de planificación

En función de las características de los procesos que se ejecutan en el sistema convendrá hacer uso de una u otra política de planificación. Veamos las más empleadas.

### 4.1 Primero en llegar primero en ser servido, o FCFS (del inglés, *First Come First Served*)

Se emplea en procesos por lotes (sin intervención del usuario) y es *no apropiativo*. Los procesos se van poniendo en cola según llegan y se les asigna el estado *Preparado*. Cuando es asignado al procesador, no lo abandona hasta que termina.

Una de sus ventajas principales consiste en que es un algoritmo muy sencillo de implementar y también es fácilmente predecible.

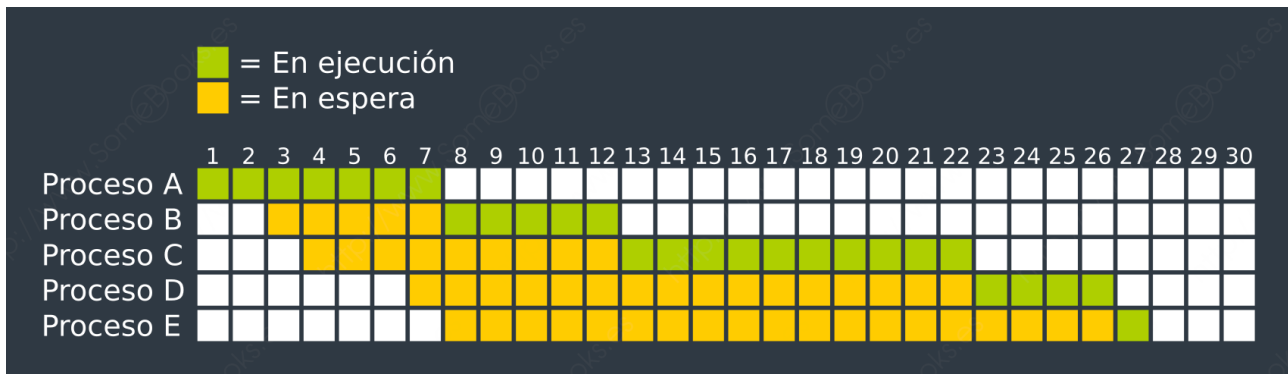
Entre sus principales inconvenientes podemos mencionar que los procesos largos pueden hacer esperar mucho a los procesos cortos y que el tiempo de servicio mínimo variará mucho según el número de procesos ejecutados y la duración de los mismos. Además, el tiempo medio de espera depende de que lleguen antes los procesos más cortos o los más largos.

Para entender su funcionamiento, podemos estudiar el siguiente ejemplo práctico en el que disponemos de cinco procesos que se incorporan al sistema en momentos sucesivos y que tienen diferentes tiempos de duración:

## Planificación FCFS

| Proceso | Momento de llegada | Duración |
|---------|--------------------|----------|
| A       | 1                  | 7        |
| B       | 3                  | 5        |
| C       | 4                  | 10       |
| D       | 7                  | 4        |
| E       | 8                  | 1        |

Si analizamos de forma gráfica el comportamiento del sistema, observaremos que se comporta del siguiente modo.



En consecuencia, podemos extraer las siguientes conclusiones:

| Planificación FCFS |                    |          |                     |                        |                     |                  |
|--------------------|--------------------|----------|---------------------|------------------------|---------------------|------------------|
| Proceso            | Momento de llegada | Duración | Momento de comienzo | Momento de terminación | Tiempo de respuesta | Tiempo de espera |
| A                  | 1                  | 7        | 1                   | 7                      | $7-0 = 7$           | 0                |
| B                  | 3                  | 5        | 8                   | 12                     | $12-2 = 10$         | $10-5 = 5$       |
| C                  | 4                  | 10       | 13                  | 22                     | $22-3 = 19$         | $19-10 = 9$      |
| D                  | 7                  | 4        | 23                  | 26                     | $26-6 = 20$         | $20-4 = 16$      |
| E                  | 8                  | 1        | 27                  | 33                     | $33-7 = 26$         | $26-1 = 25$      |

#### 4.1.1 Actividad

Supón que tenemos tres usuarios y que cada uno necesita ejecutar un proceso por lotes distinto:

- El primero necesitará 18 minutos para ejecutarse.
- El segundo necesitará 6 minutos.
- El tercero otros 7 minutos.

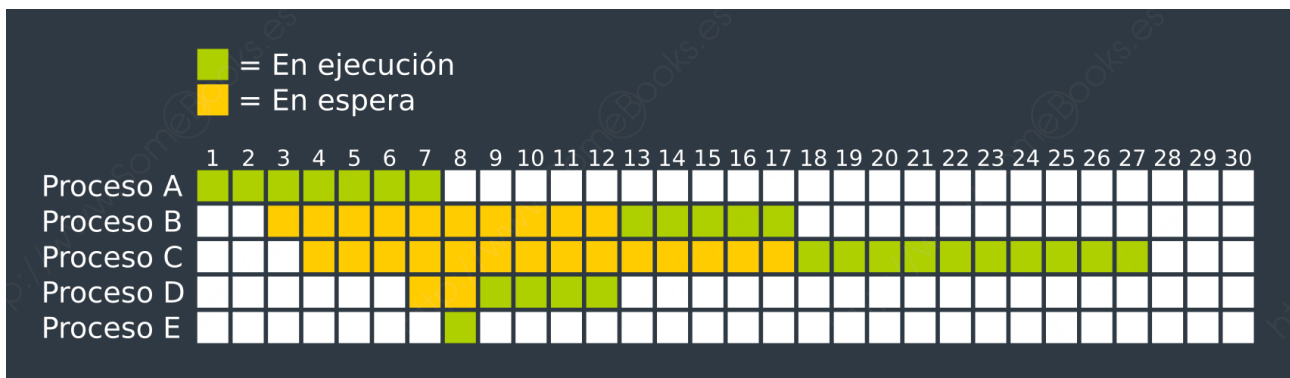
Sabiendo que el tiempo medio de respuesta será la suma de los tiempos que espera cada usuario para obtener sus resultados, dividido entre tres, ¿tendrá alguna importancia el orden en el que se planifiquen los procesos si utilizamos un algoritmo FCFS y sólo disponemos de un procesador?

Para ilustrar tu conclusión, deberás crear gráficas con las diferentes combinaciones y calcular el tiempo medio de respuesta en cada caso.

## 4.2 Primero el más corto, o SJF (del inglés, *Shortest Job First*)

Se emplea en procesos por lotes (sin intervención del usuario) y es *no apropiativo*. Los procesos se van poniendo en cola según llegan y se les asigna el estado *Preparado*, pero el *Planificador* elige el que tiene un menor tiempo previsto de ejecución.

Como en el caso anterior, podemos estudiar un ejemplo práctico para entender su funcionamiento. En este caso, partiremos de los mismos datos, para que puedas comparar los resultados:



Y analizando los resultados, obtendríamos la siguiente tabla:

| Planificación SJF |                    |          |                     |                        |                     |                  |
|-------------------|--------------------|----------|---------------------|------------------------|---------------------|------------------|
| Proceso           | Momento de llegada | Duración | Momento de comienzo | Momento de terminación | Tiempo de respuesta | Tiempo de espera |
| A                 | 1                  | 7        | 1                   | 7                      | $7-0 = 7$           | 0                |
| B                 | 3                  | 5        | 13                  | 17                     | $17-2 = 15$         | $15-5 = 10$      |
| C                 | 4                  | 10       | 18                  | 27                     | $27-3 = 24$         | $24-10 = 14$     |
| D                 | 7                  | 4        | 9                   | 12                     | $12-6 = 6$          | $6-4 = 2$        |
| E                 | 8                  | 1        | 8                   | 8                      | $8-7 = 1$           | $1-1 = 0$        |

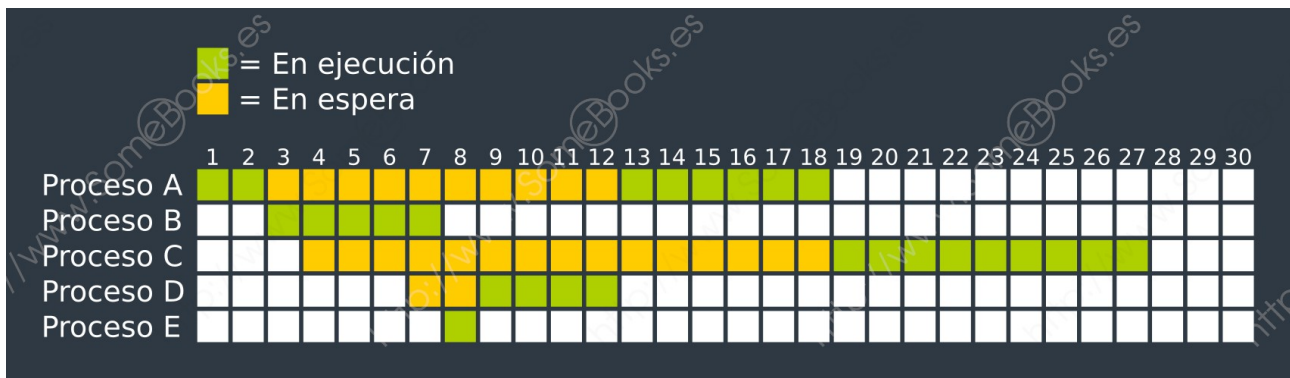
Existe una versión *apropiativa* de este algoritmo denominada *Primero el de menor tiempo restante*, o *SRTN* (del inglés, *Shortest Remaining Time Next*). En este caso, si un proceso bloqueado pasa al estado *Preparado*, el distribuidor comprueba si su tiempo restante es inferior que el del proceso que se encuentra en ejecución. En caso afirmativo, éste toma el control del procesador y el proceso que está ejecutándose pasa al estado *Preparado*.

Aunque es un algoritmo muy eficaz para los procesos cortos, resulta difícil predecir los intervalos de asignación del procesador e, incluso, puede haber procesos largos que sufran de inanición, es decir, que no lleguen a ejecutarse mientras existan procesos cortos esperando turno.

Como antes, lo ilustraremos con un ejemplo. En este caso, hemos cambiado ligeramente los datos iniciales para que sea más ilustrativo.

| Planificación SRTN |                    |          |
|--------------------|--------------------|----------|
| Proceso            | Momento de llegada | Duración |
| A                  | 1                  | 8        |
| B                  | 3                  | 5        |
| C                  | 4                  | 9        |
| D                  | 7                  | 4        |
| E                  | 8                  | 1        |

Y el gráfico resultante quedará como sigue:



De nuevo, aquí tienes los datos arrojados, representados en forma de tabla:

| Planificación SRTN |                    |          |                     |                        |                     |                  |
|--------------------|--------------------|----------|---------------------|------------------------|---------------------|------------------|
| Proceso            | Momento de llegada | Duración | Momento de comienzo | Momento de terminación | Tiempo de respuesta | Tiempo de espera |
| A                  | 1                  | 8        | 1                   | 18                     | $18 - 0 = 18$       | $18 - 8 = 10$    |
| B                  | 3                  | 5        | 3                   | 7                      | $7 - 2 = 5$         | $5 - 5 = 0$      |
| C                  | 4                  | 9        | 19                  | 27                     | $27 - 3 = 24$       | $24 - 9 = 15$    |
| D                  | 7                  | 4        | 9                   | 12                     | $12 - 6 = 6$        | $6 - 4 = 2$      |
| E                  | 8                  | 1        | 8                   | 8                      | $8 - 7 = 1$         | $1 - 1 = 0$      |

#### 4.2.1 Actividad: Planificación SRTN

Supón que tenemos tres usuarios y que cada uno necesita ejecutar un proceso por lotes distinto:

- El primero necesitará 18 minutos para ejecutarse, y es el único que está listo al principio.
- El segundo necesitará 10 minutos y está disponible a partir del minuto 7.
- El tercero necesitará 6 minutos y está disponible a partir del minuto 21.

A partir de estos datos, crea una gráfica donde se aprecie el orden de ejecución de los procesos suponiendo que usamos el algoritmo *SRTN* si el *Planificador* toma el control del sistema una vez por minuto (y desestimamos el tiempo que se está ejecutando).

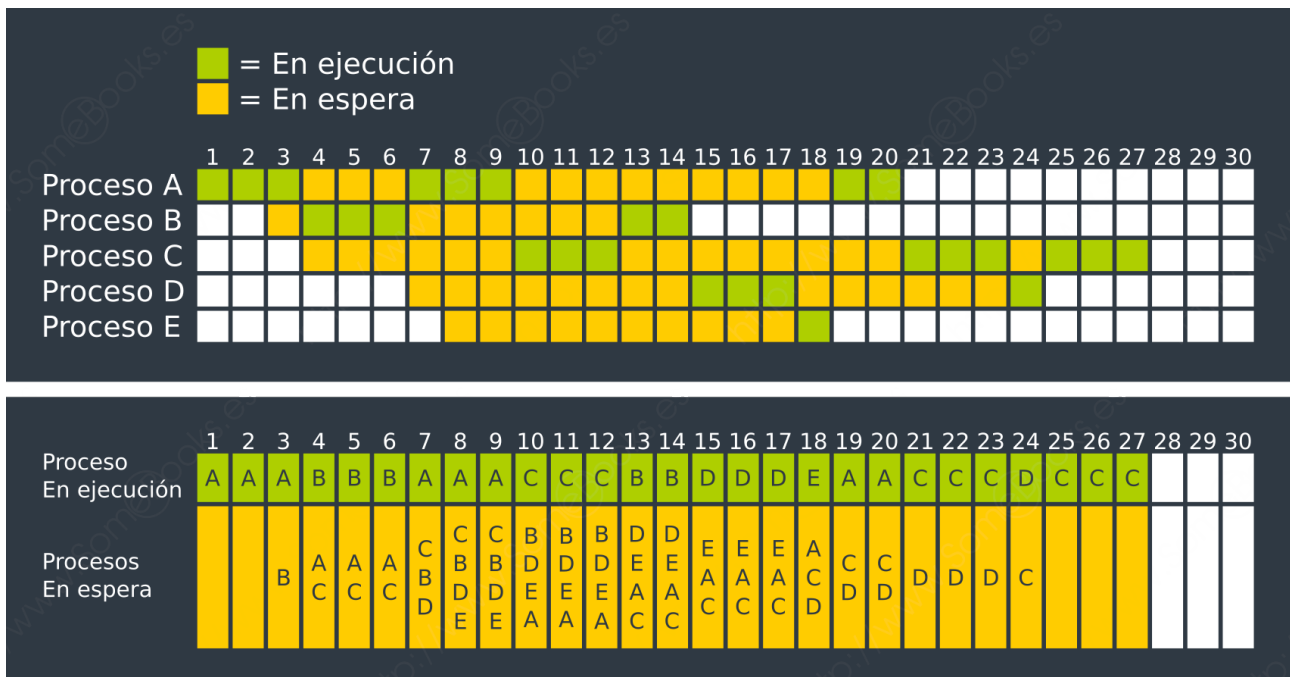
A continuación, crea una tabla donde expreses el tiempo de respuesta y el tiempo de espera en función de los datos de entrada.

### 4.3 Por turnos, o RR (del inglés, *Round Robin*)

Se emplea en procesos interactivos (en los que interviene el usuario) y es *apropiado*. Los procesos se van poniendo en cola según llegan y se les asigna el estado *Preparado*. El procesador se irá asignando a cada proceso, por orden, durante una fracción de tiempo llamada *Quantum*, que es igual para todos. Si el proceso se acaba, se bloquea, o si se agota su tiempo, el procesador es liberado para el siguiente proceso de la lista.

Por lo tanto, la cola de procesos actúa como una estructura circular con organización FIFO.

Para ilustrarlo, comenzaremos por los mismos datos de entrada del ejemplo anterior, suponiendo un *Quantum* equivalente a tres unidades de tiempo. El resultado sería como el que muestra la siguiente imagen:



Para facilitar la comprensión, hemos incluido también una representación gráfica de los procesos que se encuentran en cola para ser ejecutados y del proceso que se encuentra en ejecución en cada unidad de tiempo.

Finalmente, la siguiente tabla muestra los datos que ofrece:

| Planificación Round Robin |                    |          |                     |                        |                     |                  |
|---------------------------|--------------------|----------|---------------------|------------------------|---------------------|------------------|
| Proceso                   | Momento de llegada | Duración | Momento de comienzo | Momento de terminación | Tiempo de respuesta | Tiempo de espera |
| A                         | 1                  | 8        | 1, 7, 19            | 20                     | $20-0=20$           | $20-8=12$        |
| B                         | 3                  | 5        | 4, 13               | 14                     | $14-2=12$           | $12-5=7$         |
| C                         | 4                  | 9        | 10, 21, 25          | 27                     | $27-3=24$           | $24-9=15$        |
| D                         | 7                  | 4        | 15, 24              | 24                     | $24-6=18$           | $18-4=14$        |
| E                         | 8                  | 1        | 18                  | 18                     | $18-7=11$           | $11-1=10$        |

#### 4.3.1 Actividad: Planificación por turnos

Partiendo de los mismos datos de entrada que en la actividad anterior, aplica la *planificación por turnos* suponiendo un Quantum equivalente a tres unidades de tiempo. Debes realizar las siguientes tareas:

1. Crea un esquema, como el aportado en el capítulo, pero teniendo en cuenta los ‘Momentos de llegada’ y la ‘Duración’ de los procesos de la actividad anterior.
2. A continuación, crea una nueva versión de la tabla de planificación, ajustando su contenido al diseño del esquema creado en el apartado anterior.

## 4.4 Planificación por prioridad

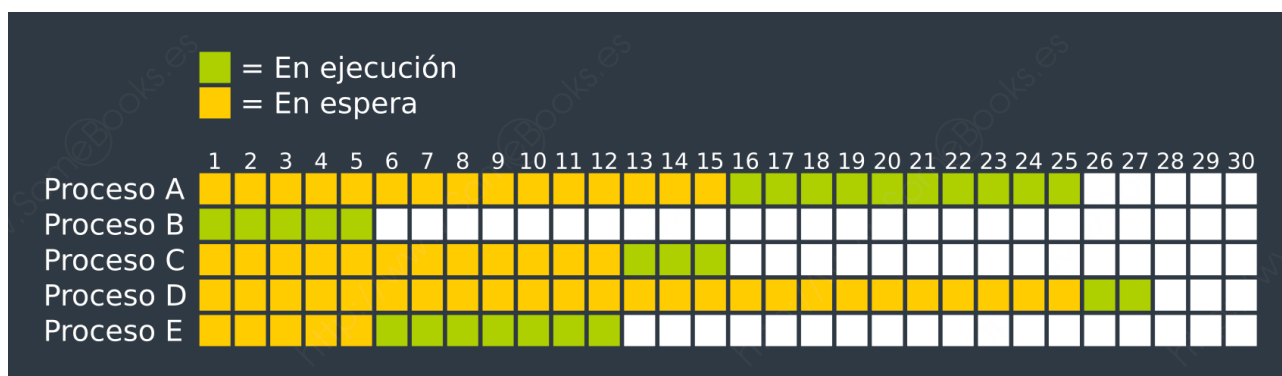
Se emplea en procesos interactivos (en los que interviene el usuario) y es *apropiativo*. A cada proceso se le asigna un número entero que representa su prioridad, de modo que, cuanto menor es el número, mayor es la prioridad.

Si se considera un algoritmo *no apropiativo*, funcionaría como el algoritmo *Primero el más corto* (SJF), pero considerando la prioridad en lugar de la duración.

Aún así, veamos un ejemplo, suponiendo que todos los procesos están disponibles desde el primer momento. Estos serían nuestros datos de partida:

| Planificación por prioridad |          |           |
|-----------------------------|----------|-----------|
| Proceso                     | Duración | Prioridad |
| A                           | 10       | 4         |
| B                           | 5        | 1         |
| C                           | 3        | 3         |
| D                           | 2        | 5         |
| E                           | 7        | 2         |

Siguiendo la asignación del procesador según el orden de prioridad (el más bajo primero), obtendríamos el siguiente gráfico:



Finalmente, en la siguiente tabla recogemos los datos numéricos obtenidos:

| Planificación por prioridad |          |           |                     |                        |                     |                  |
|-----------------------------|----------|-----------|---------------------|------------------------|---------------------|------------------|
| Proceso                     | Duración | Prioridad | Momento de comienzo | Momento de terminación | Tiempo de respuesta | Tiempo de espera |
| A                           | 10       | 4         | 16                  | 25                     | 25                  | $25 - 10 = 15$   |
| B                           | 5        | 1         | 1                   | 5                      | 5                   | $5 - 5 = 0$      |
| C                           | 3        | 3         | 13                  | 15                     | 15                  | $15 - 3 = 12$    |
| D                           | 2        | 5         | 26                  | 27                     | 27                  | $27 - 2 = 25$    |
| E                           | 7        | 2         | 6                   | 12                     | 12                  | $12 - 7 = 5$     |

Para evitar que los procesos con una prioridad más elevada monopolicen el uso del procesador (y los de prioridad más baja padezcan inanición), la prioridad de los procesos puede ir reduciéndose cada vez que obtengan el uso del procesador. Así, se convertiría en una versión *apropiativa* que funcionaría como *Primero el de menor tiempo restante (SRTN)*, pero tomando la prioridad en lugar de la duración y disminuyéndola cada vez.

## 5. Comandos linux que interactúan con el planificador de procesos

### 5.1 Comando "ps" - Visualización de Procesos

El comando "ps" se utiliza para mostrar información sobre los procesos en ejecución. Puedes obtener detalles como el ID del proceso, el estado, el uso de recursos y mucho más. Aquí hay dos ejemplos de uso de "ps":

**Ejemplo 1:** Listar todos los procesos en ejecución de la sesión actual.

*ps*

**Ejemplo 2:** Listar todos los procesos en ejecución en el sistema.

*ps aux*

**Ejemplo 3:** Mostrar solo los procesos de un usuario específico (reemplaza "nombre\_de\_usuario" por el nombre de usuario real).

*ps -u nombre\_de\_usuario*

**Ejemplo 4:** Ver procesos en formato de árbol, mostrando las relaciones de padre e hijo:

*ps aux --forest*

**Ejemplo 5:** Listar procesos por su identificador de proceso (PID):

*ps -p PID*

**Ejemplo 6:** Mostrar información detallada sobre un proceso específico, incluyendo uso de CPU y memoria:

*ps -p PID -o pid,ppid,%cpu,%mem,cmd*

**Ejemplo 7:** Mostrar todos los procesos incluyendo los de otros usuarios:

*ps -e*

**Ejemplo 8:** Mostrar procesos en un formato personalizado, especificando las columnas deseadas (por ejemplo, PID, nombre del proceso y uso de CPU):

*ps -e -o pid,comm,%cpu*

**Ejemplo 9:** Listar los procesos ordenados por uso de CPU en orden descendente:

*ps aux --sort=-%cpu*

**Ejemplo 9:** Mostrar el uso de memoria de los procesos ordenados por el mayor uso de memoria en orden descendente:

*ps aux --sort=-%mem*

### 5.1.1 Actividades

1. ¿Cuál es el propósito principal del comando `ps` en Unix y Linux?
2. ¿Cómo se lista todos los procesos en ejecución en el sistema?
3. ¿Qué comando `ps` se utiliza para mostrar información detallada de todos los procesos, incluyendo los de otros usuarios?
4. ¿Cómo se muestra la lista de procesos en formato de árbol con las relaciones de padre e hijo?
5. ¿Cómo se ordenan los procesos por uso de CPU en orden ascendente?
6. ¿Cuál es la diferencia entre las banderas `-e` y `-A` en el comando `ps`? Escribe un ejemplo de ambos uso comprobando su funcionamiento.
7. ¿Qué bandera se utiliza para mostrar todos los procesos en un formato personalizado?
8. ¿Cómo se muestra el uso de memoria de los procesos ordenados por el mayor uso de memoria en orden descendente?
9. ¿Cuál es la diferencia entre `ps -f` y `ps aux`? Prueba el funcionamiento de ambos comandos
10. ¿Cómo se lista solo tus propios procesos en ejecución?
11. ¿Cómo se muestra la hora de inicio de los procesos en la salida de `ps`?
12. ¿Cuál es la columna que representa el identificador de proceso (PID) en la salida de `ps`?
13. ¿Qué bandera se utiliza para mostrar la información de CPU y memoria de un proceso específico?
14. ¿Cómo se filtran los procesos que contienen un nombre específico en el comando `ps`? Escribe y ejecuta el comando correspondiente.
15. ¿Cuál es el propósito de la bandera `--sort` en `ps` y cómo se utiliza? Escribe y ejecuta un ejemplo de este comando
16. ¿Cuál es la diferencia entre `ps -o` y `ps -O`? Comprueba la ejecución de ambos comandos.
17. ¿Qué página de manual puedes consultar para obtener más información sobre el comando `ps`?

### 5.1.2 Actividades

1. Ejecuta el comando "ps" en tu sistema y lista todos los procesos en ejecución.
2. Utiliza el comando "ps" para listar los procesos que están utilizando más del 50% de la CPU.
3. Filtra y muestra solo los procesos que pertenecen a un usuario específico.
4. Con el comando pgrep puedes encontrar el PID de un proceso con un determinado nombre: pgrep apache. O:
5. Con la ayuda del comando pgrep, encuentra el ID de proceso (PID) de un proceso que quieras y obtén información detallada sobre él con "ps".
6. Crea un script de bash que ejecute un proceso y usa "ps" para verificar su estado periódicamente.
7. Crea el siguiente script de linux (denominado *sc1*) que cada 5 segundos muestra los estados de los procesos pertenecientes a la sesión que estés utilizando:

```
pid=$!
```

```
Echo "Proceso iniciado con el PID: $pid"
```

```
while true; do
```

```
proceso_info=$(ps -aux)
```

```
echo "Estado del proceso:"
```

```
echo "$proceso_info"
```

```
sleep 5
```

```
done
```

8. Ejecuta el script anterior (*./sc1*). Si tienes problemas para ejecutarlo dále permisos totales: *chmod 777 pr1*.
9. Modifica el código del script anterior (guárdalo con el nombre de *sc2*) para que la muestra de los estados de los procesos pertenecientes a la sesión que haces uso, se realice cada 6 segundos.
10. Modica el script anterior (guárdalo con el nombre de *sc3*) para que la información sobre el estado de los procesos sea la de todo el sistema.
11. Haciendo uso del comando ps en alguno de los ejemplos comentados en el apartado anterior. Implementa un nuevo script que vaya dando información del estado de los procesos del sistema cada 7 segundos.

## 5.2 Comando "kill" - Terminación de Procesos

El comando "kill" se utiliza para enviar señales a los procesos y, por lo general, se usa para detenerlos. Aquí tienes dos ejemplos de uso de "kill":

**Ejemplo 1:** Terminar un proceso por su ID de proceso (PID).

*kill PID*

**Ejemplo 2:** Forzar la terminación de un proceso que no responde.

*kill -9 PID*

### 5.2.1 Actividades

1. Encuentra un proceso en tu sistema que no esté respondiendo y utilice "kill" para forzar su terminación.
2. Crea un proceso simple en segundo plano y luego usa "kill" para detenerlo por su PID.
3. Escoge un proceso y utilice "kill" para enviar una señal diferente (por ejemplo, SIGSTOP) para ponerlo en pausa y luego otra señal para reanudarlo.

### 5.3. Comando "top" - Monitoreo en Tiempo Real

El comando "top" se utiliza para monitorear los procesos en tiempo real y ver su uso de recursos: CPU, la memoria y más.

**Ejemplo 1:** Ejecutar "top" para ver los procesos en tiempo real.

*Top*

**Ejemplo 2:** Ordenar los procesos por uso de CPU.

*top -o %CPU*

### 5.3.1 Actividades

1. Ejecuta "top" y observa los procesos en tiempo real. Identifica los procesos que consumen más CPU y memoria.
2. ¿Qué diferencia hay entre el uso del comando ps y el comando top ?
3. Utiliza "top" para ordenar los procesos por uso de memoria.
4. Limita la salida de "top" para que solo muestre un número específico de procesos.
5. Actualiza la pantalla de "top" cada 3 segundos y observa cómo cambian los valores.
6. Encuentra un proceso con alto uso de CPU en "top" y utiliza "kill" para detenerlo.
7. El comando top va mostrando un número indefinido de instantáneas en tiempo real sobre los procesos del sistema. Ejecuta el comando top -n5, comprueba que el número total de instantáneas mostrando información sobre los estados de los procesos se limita a 5.
- 8.

En un sistema operativo GNU/Linux, ejecuta la siguiente orden,

**\$ top -n 1**

que lanza una única iteración de top.  
Identifica los siguientes datos:

1. ¿Cuánto tiempo lleva encendido el ordenador?
2. ¿Cuántos procesos se encuentran ejecutándose en el sistema? De ellos, ¿cuántos están en ejecución, parados, en espera o zombis?
3. ¿Cómo se distribuye el porcentaje de uso de la CPU?
4. ¿Cuál el proceso que más CPU está consumiendo? ¿Qué características tiene este proceso?

## 5.4 Comando pstree

Muestra una representación jerárquica de todos los procesos en ejecución. Se puede ver en ella la relación jerárquica entre los procesos (padres, hijos, ...).

### 5.4.1 Actividades

#### 1. Visualizar la jerarquía de procesos del sistema:

- Ejecuta `ps tree` en la terminal para obtener una visión general de la jerarquía de procesos en tu sistema. Observa cómo se presentan los procesos y cómo están interrelacionados.

#### 2. Mostrar procesos de un usuario específico:

- Utiliza `ps tree -u <nombre_usuario>` para ver la jerarquía de procesos de un usuario específico. Observa cómo se relacionan los procesos con el usuario especificado y cómo se organizan en la estructura del árbol.

#### 3. Explorar procesos de un servicio en particular:

- Ejecuta `ps tree | grep <nombre_servicio>` para ver los procesos relacionados con un servicio específico. Analiza cómo se conectan los procesos del servicio y cómo interactúan con otros procesos en el sistema.

#### 4. Mostrar procesos hijos de un proceso en particular:

- Ejecuta `ps tree -p <PID>` para ver los procesos hijos del proceso con un ID de proceso (PID) específico. Examina la jerarquía de procesos descendientes y comprende la relación entre el proceso principal y sus procesos secundarios.

#### 5. Visualizar procesos con detalles de comandos:

- Utiliza `ps tree -a` para ver la jerarquía de procesos junto con los detalles de los comandos ejecutados por cada proceso. Observa los comandos exactos que se están ejecutando en cada rama del árbol de procesos y comprende cómo están relacionados.

## 5.5. [pgrep](#)

Devuelve los IDs de proceso (PIDs) que coinciden con el criterio de búsqueda especificado.

Ejemplo:

*[pgrep](#) [firefox](#)*

### 5.5.1 Actividades

#### **Identificar procesos asociados con un usuario específico:**

- Usa `pgrep -u <nombre_usuario>` para encontrar todos los IDs de procesos asociados con un usuario específico. Examina los procesos en ejecución para ese usuario y realiza un análisis del comportamiento del sistema.

#### **2. Buscar procesos que contienen un patrón en su nombre:**

- Ejecuta `pgrep -l <patrón>` para encontrar todos los procesos cuyos nombres coinciden con un patrón específico. Identifica procesos con un nombre particular y entiende cómo se están ejecutando en el sistema.

#### **3. Encontrar el PID de un proceso específico:**

- Utiliza `pgrep <nombre_proceso>` para obtener el ID de proceso (PID) de un proceso específico. Luego, puedes utilizar este PID para realizar operaciones adicionales, como finalizar el proceso con el comando `kill`.

#### **4. Verificar la existencia de un proceso en ejecución:**

- Emplea `pgrep -x <nombre_proceso>` para comprobar si un proceso específico está en ejecución en el sistema. Observa si el proceso está activo y en funcionamiento y toma medidas en consecuencia.

#### **5. Analizar la salida detallada de pgrep junto con ps:**

- Combina `pgrep` con `ps` para obtener más información sobre un proceso en particular. Usa `pgrep <nombre_proceso>` y luego pasa el PID obtenido a `ps` para ver detalles como el uso de la CPU, la memoria y otros recursos del proceso identificado.

## 5.6 nice y renice

**nice:** Ejecuta un proceso con una prioridad modificada, ajustando su asignación de tiempo de CPU.

Ejemplo:

***nice -n 10 firefox***

Se debe de tener en cuenta que un valor de nice negativo es más prioritario que un valor de nice positivo.

**renice:** Cambia la prioridad de un proceso en ejecución.

Ejemplo:

***renice -n 10 -p 1234***

1234 es el pid del proceso que se le va a cambiar la prioridad

### 5.6.1 Actividades

1. Ejecutar un proceso con diferentes niveles de prioridad.
2. Comparar el rendimiento del sistema con y sin el comando **nice**:
  - Ejecuta una aplicación que consuma muchos recursos sin usar nice, y luego vuelve a ejecutarla con un nivel de prioridad modificado utilizando nice. Mide y compara el rendimiento del sistema en ambos casos.
3. Modificar la prioridad de un proceso en ejecución.

Ejecuta un proceso intensivo en la CPU y utiliza renice para ajustar su prioridad en tiempo real. Observa cómo cambia el rendimiento del proceso y del sistema en general.

## 5.7 Modificación de la política de planificación

- Especificación de una política de planificación FIFO con prioridad 80  
*chrt -f -p 80 <PID>*
- Especificación de una política de planificación Round-robin con prioridad 70  
*chrt -r -p 70 <PID>*
- Volver a política normal (CFS)  
*chrt -o -p 0 <PID>*